

DEVELOPER.md

Quick orientation for the **MEYcell** wiki (McMaster iGEM 2026). Goal of this doc: get you writing page content in ~5 minutes.

0. Before getting started

Each of you should work in your own branches. Please run the following following commands:

- Vaish: `git checkout vaish`
- Amber: `git checkout amber`
- Alexis: `git checkout alexis`

Make sure this operation is successful and you haven't received any error messages. Contact me if you did.

1. Run it

```
pnpm install
pnpm dev      # http://localhost:3000
pnpm build    # production build + typecheck
pnpm lint     # Biome check
pnpm format   # Biome format --write
```

pnpm only. Biome (not ESLint/Prettier) does linting + formatting.

2. Where things live

```
src/
├─ app/                    # routes — one folder per page
│   └─ page.tsx            # home (landing sections)
│   └─ team/ project/ engineering/
│   └─ documentation/ community/ entrepreneurship/
│   └─ layout.tsx          # Header + Footer shell
│   └─ globals.css         # ← color system (single source of truth)
├─ components/
│   └─ wiki/               # the content-authoring kit (Section 3)
│       └─ main/           # home-page-only sections
│       └─ layout/         # Header, Footer
│       └─ ui/             # shared primitives
├─ lib/wiki/               # citations, nav links, card colors
└─ types/wiki.ts          # shared types
```

Rule of thumb: **static content lives in the file that uses it.** There is no global content module — copy, stats, and lists sit next to the component that renders them.

3. Writing a page (the important part)

Every content page is the same three-part shape: a `WikiPage` wrapper, some `WikiSection` blocks, and an optional `ReferencesSection`.

```
import { ReferencesSection } from "@components/wiki/ReferencesSection";
import { WikiPage } from "@components/wiki/WikiPage";
import { WikiSection } from "@components/wiki/WikiSection";
import { createCitations } from "@lib/wiki/citations";

export const metadata = { title: "Project — MEYcell" };

// One ordered list = the page's sources. Order sets the [1], [2] numbering.
const { Cite, references } = createCitations([
  { id: "koch2019", authors: "Koch, B. et al.", title: "...",
    source: "Metabolic Engineering", year: 2019, url: "https://..." },
]);

export default function ProjectPage() {
  return (
    <WikiPage eyebrow="Project" title="The MEYcell Project" intro="One-line summary.">
      <WikiSection id="overview" title="Overview">
        <p>Any JSX you like<Cite id="koch2019" />.</p>
      </WikiSection>

      <WikiSection id="results" title="Results">
        <p>...</p>
      </WikiSection>

      <ReferencesSection id="references" title="References" references={references} />
    </WikiPage>
  );
}
```

To add content, add a `<WikiSection>`. That's it — the sidebar table of contents builds itself from the sections you write.

The pieces

Component	Props	What it does
<code>WikiPage</code>	<code>eyebrow</code> , <code>title</code> , <code>intro?</code> , <code>children</code>	Page shell. Auto-builds the TOC sidebar from its section children.
<code>WikiSection</code>	<code>id</code> , <code>title</code> , <code>children</code>	One content block → <code><h2></code> + your JSX. <code>id</code> is its anchor.
<code>ReferencesSection</code>	<code>id</code> , <code>title</code> , <code>references</code>	A <code>WikiSection</code> specialization. Place it manually (usually last); still appears in the TOC.
<code>createCitations(refs)</code>	array of <code>WikiReference</code>	Returns <code>{ Cite, references }</code> .
<code>Cite</code>	<code>id</code>	Inline <code>[n]</code> marker linking to its reference.
<code>Banner</code>	<code>src?</code> , <code>alt?</code> , <code>eyebrow?</code> , <code>title</code> , <code>children?</code>	Full-width page header w/ image. No <code>src</code> → compact header.
<code>PagePlaceholder</code>	<code>eyebrow</code> , <code>title</code> , <code>description</code>	"Coming soon" stub for unbuilt pages.

Rules that matter

- **Sections must be direct children of `WikiPage`.** Don't wrap them in a `<div>` or a `.map()` — the TOC reads their props directly and will miss them.
- **`id` must be unique per page** (it's the anchor target).
- **Citation numbering = array order.** Reorder the array, both the inline markers and the reference list renumber together. Keep `ids` unique.
- Adding a new section *type* that should appear in the TOC? Register it in `SECTION_TYPES` in `WikiPage.tsx`.

Adding a new page

1. `src/app/<name>/page.tsx` → `export metadata` + a default component.
2. Add it to `NAV_LINKS` in `src/lib/wiki/nav.ts` (drives header, mobile menu, and the home CTA row).
3. Add it to the footer lists in `src/components/layout/Footer.tsx` if relevant.

4. Colors

`src/app/globals.css` is the single source of truth. Raw hex → semantic `@theme` tokens → Tailwind utilities. Never hardcode a color in a component.

Use	Utility	Hex
Page background	<code>bg-background</code>	<code>#071a2e</code>
Section band	<code>bg-section</code>	<code>#060f1e</code>
Card	<code>bg-card</code>	<code>#070e1c</code>
Footer	<code>bg-footer</code>	<code>#040b14</code>
Brand teal	<code>text-primary</code> / <code>bg-primary</code>	<code>#00d4b4</code>
Teal dark	<code>text-accent</code>	<code>#00b89a</code>
Borders	<code>border-border</code>	<code>#0d2d4a</code>

Text ramp, lightest → faintest: `text-foreground #d8eef8` · `text-body #9ac8e0` · `text-muted-foreground #6ba8c4` · `text-dim #5d8faa` · `text-faint #3d6a80` · `text-decorative #2a4a5c`

For translucent glows/borders use `rgb(var(--teal-rgb) / 0.08)` so the hue still comes from the one source.

Fonts: `font-sans`, `font-mono`, `font-display` (currently `font-display` aliases Geist Sans — repoint `--font-display` in `globals.css` to change it).

5. Conventions

- **React Compiler is on** — no `useMemo` / `useCallback` / `React.memo` unless there's a documented reason.
- **No `any`**. Shared types go in `src/types/wiki.ts`.
- Keep page files thin, components small, JSX shallow.
- Please only work within the files your tasks were assigned.

Before you push: `pnpm lint` · `pnpm build`.